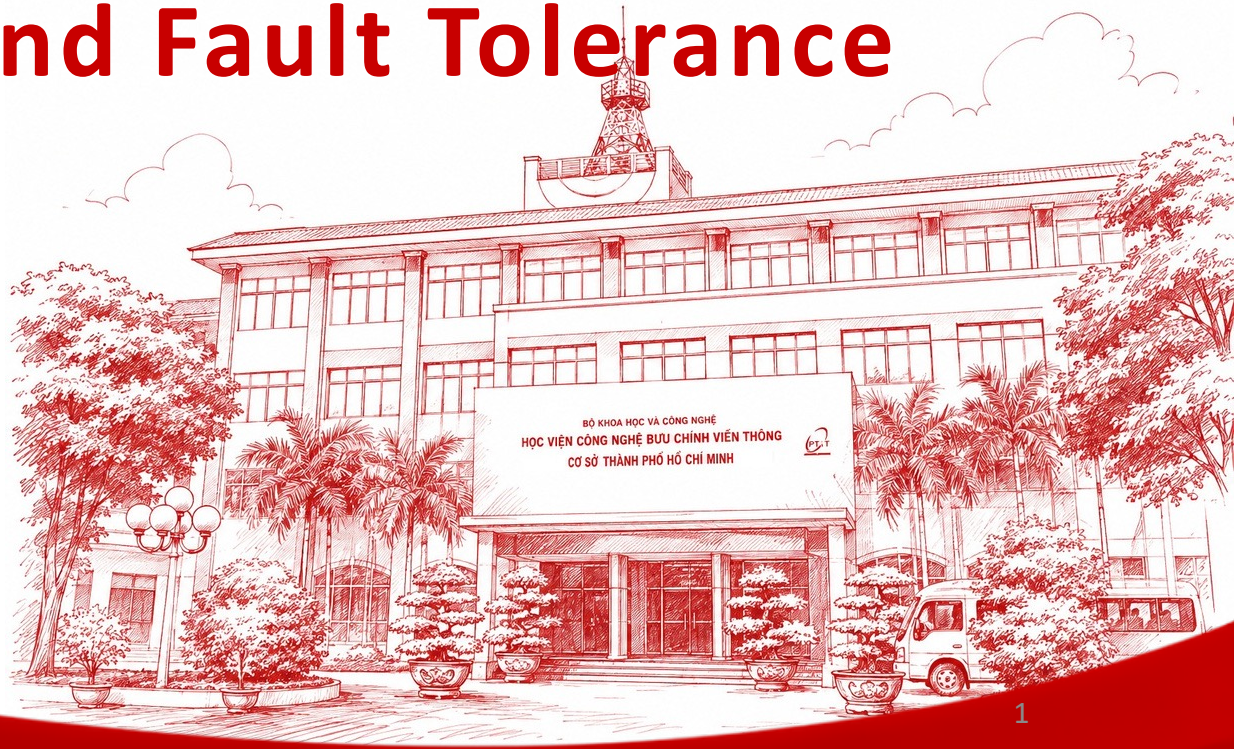
INT1405 - DISTRIBUTED SYSTEMS

LECTURE 05

Transaction Management, Concurrency Control, Recovery, and Fault Tolerance

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn



Tanenbaum & van Steen (2nd Edition), Chapter 1 (Sec 1.3.2) & Chapter 8 (Sec 8.1–8.6)

Learning Objectives & Roadmap

Primary Learning Goals

- Analyze distributed ACID transaction properties and nested hierarchies.
- Classify system failures into Crash-stop, Omission, Timing, and Byzantine models.
- Compute process replication bounds (k+1 vs 2k+1 vs 3k+1) for fault tolerance.
- Trace Two-Phase Commit (2PC) and Three-Phase Commit (3PC) state transitions.
- Design coordinated checkpoint schedules to eliminate the Domino Effect.

Lecture Module Progression

- Module 1: Distributed Transactions & Concurrency Control (Slides 01–08)
- Module 2: Fault Tolerance Concepts & Failure Models (Slides 09–16)
- Module 3: Reliable Communication & Distributed Commit Protocols (Slides 17–26)
- Module 4: System Recovery Mechanisms & Logging Strategies (Slides 27–34)
- Module 5: Real-World Systems, Assessment & Practical Lab (Slides 35–40)

Content

- Module 1: Distributed Transactions & Concurrency Control (Slides 01–08)
- Module 2: Fault Tolerance Concepts & Failure Models (Slides 09–16)
- Module 3: Reliable Communication & Distributed Commit Protocols (Slides 17–26)
- Module 4: System Recovery Mechanisms & Logging Strategies (Slides 27–34)
- Module 5: Real-World Systems, Assessment & Practical Lab (Slides 35–40)

Distributed Transactions & The ACID Paradigm

Transaction Definition & Scope

- An **atomic unit of execution** comprising reads, writes, and state transitions across databases.
- Execution Path: BEGIN_TRANSACTION → READ/WRITE operations → COMMIT (Durable) or ABORT (Rolled Back).
- **All-or-Nothing** Rule: Reverts to clean pre-transaction state if any sub-operation fails.

The ACID Properties

- **Atomic**: To the outside world, the transaction happens indivisibly: All operations execute successfully to completion, or the entire transaction aborts.
- **Consistent**: Preserves system invariants, schema constraints, and data integrity rules.

  - During the transaction, invariant violation may occur but is not visible outside the transaction.

- **Isolated**: Concurrent transactions do not interfere with each other. Transactions are isolated or serializable.

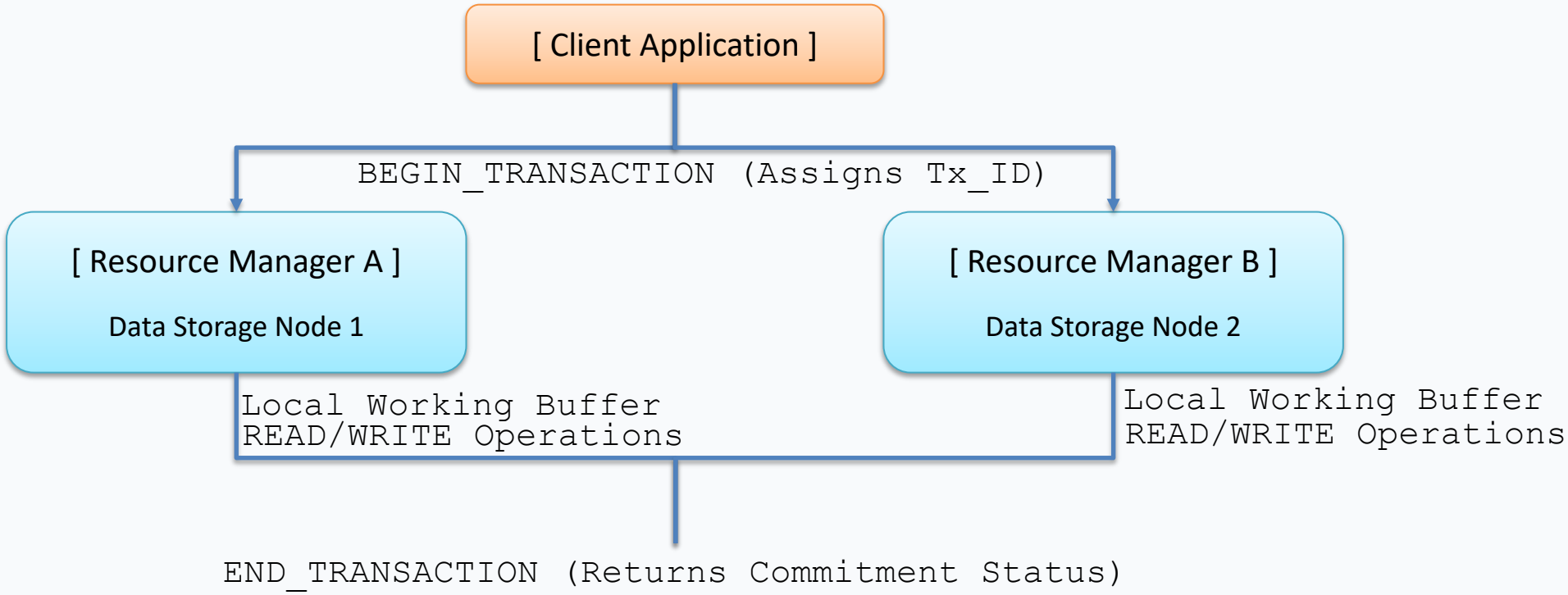
  - Intermediate uncommitted writes are invisible to concurrent transactions.

- **Durable**: Transactions are durable. Once a transaction commits, the changes are permanent.

  - Committed results survive subsequent server crashes and hardware failures.

Distributed Transactions & The ACID Paradigm

- A **distributed transaction** is an execution unit grouping multiple data manipulation operations across physically separated, networked nodes into a single, indivisible logical operation.
- Distributed Scope: Sub-operations execute across independent physical nodes connected by WAN/LAN.



Transaction Primitives & Storage System Models

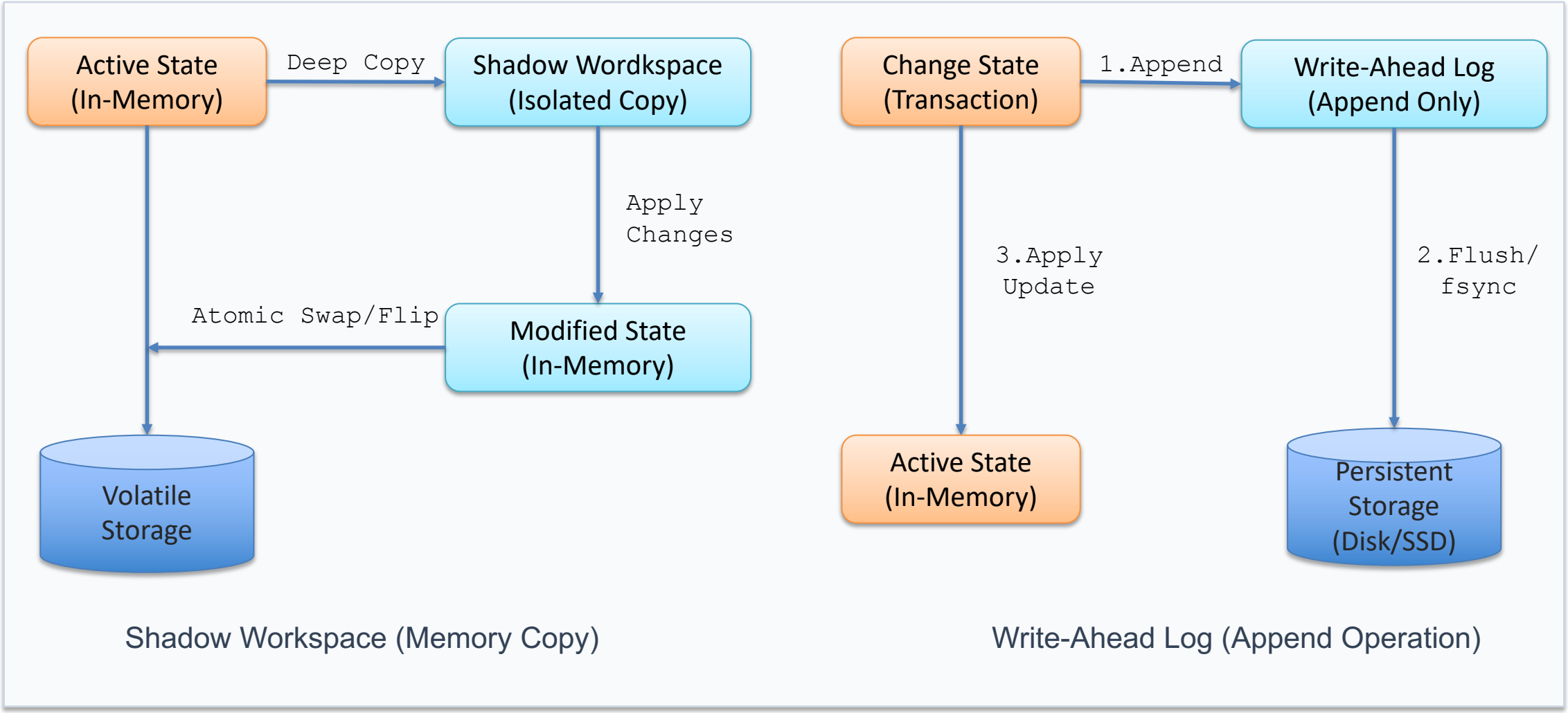
Standard Transaction Primitives

- BEGIN_TRANSACTION: Mark the opening boundary of an atomic execution block.
- END_TRANSACTION: Request completion; triggers atomic commit protocol.
- ABORT_TRANSACTION: Revert execution; undo all intermediate updates.
- READ / WRITE: Access or modify data attributes across resource managers.

Private Workspaces vs. Write-Ahead Logs

- Private Workspaces: Transaction operates on shadow copies of records, updating master data only upon commit.
- Write-Ahead Logging (WAL): Updates log intention records to stable disk storage before modifying actual data.
- WAL Dominance: Production databases use WAL to avoid expensive full-page shadow copying overhead.

Shadow Workspace vs Write-Ahead Log (WAL)



Flat Transactions vs. Nested Transactions

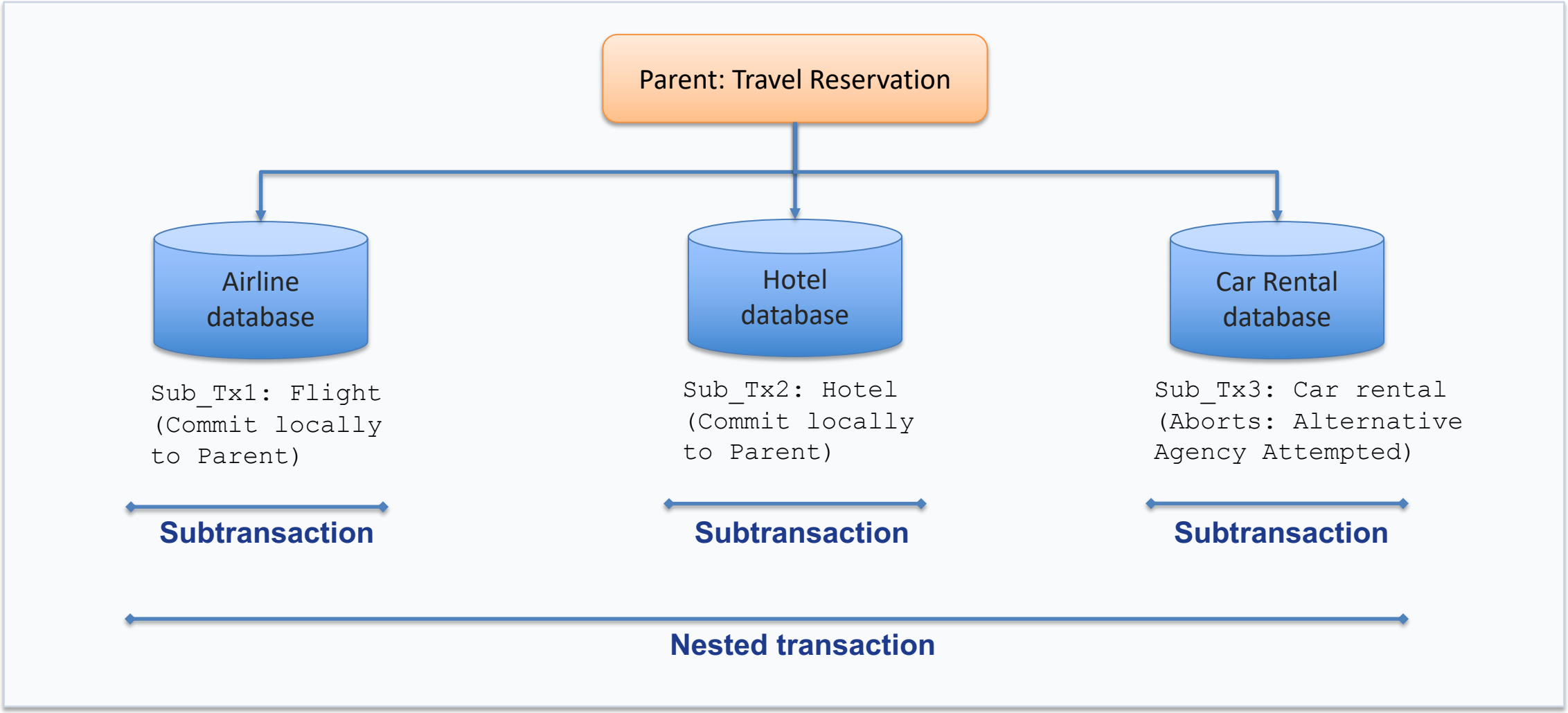
Flat Transactions

- Sequential list of operations executing between a single BEGIN and END block.
- All-or-Nothing Barrier: lack partial fault isolation. A failure at the final step forces the entire transaction to abort (full rollback to the beginning).
- Limitation: Unsuitable for long-running or distributed multi-vendor workflows.

Nested Transactions

- Hierarchical tree of subtransactions spawned by a top-level parent transaction.
- Fault Isolation: A subtransaction can abort and retry without aborting its parent.
- Conditional Commit: Subtransaction commits become permanent only when the parent commits.

Nested Transactions – Sample Application



Operational Rules of Nested Transactions

1. **Local Commit Relativity**: A subtransaction commits locally relative to its parent. Its updates become visible to its immediate parent and sibling subtransactions, but not to external top-level transactions.

2. **Top-Level Finality**: A child subtransaction's commit remains provisional until the top-level parent transaction successfully executes its final END_TRANSACTION commit. If the parent aborts, all child subtransactions abort unconditionally.

3. **Abort Containment**: Subtransaction failures are localized. If a child subtransaction aborts (e.g., Car Rental unavailable), the parent captures the exception and can launch an alternative subtransaction (e.g., query Alternative Car Rental) without aborting sibling operations (Flight or Hotel).

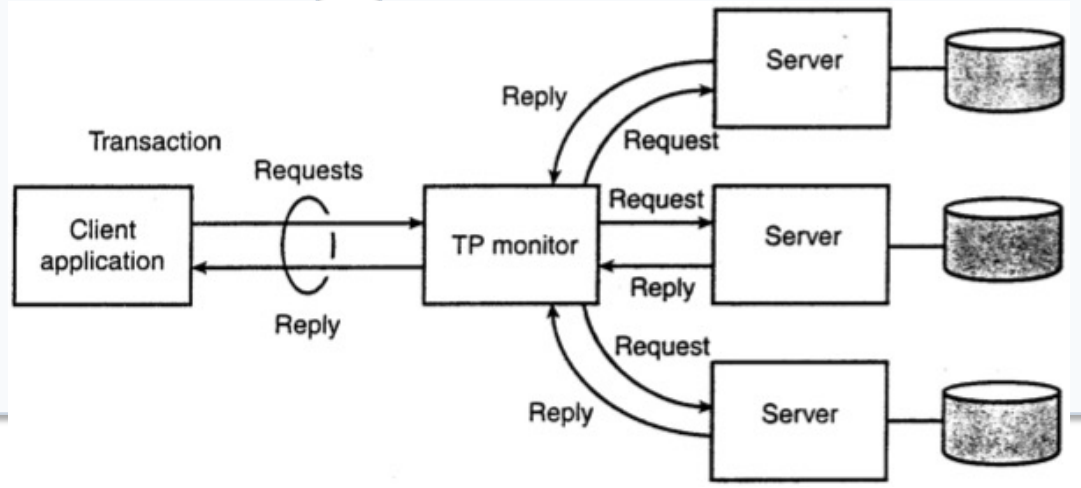
Middleware Role: Transaction Processing (TP) Monitors

Role of TP Monitors

- Specialized middleware coordinating distributed transactions across heterogeneous databases, applications, and networks.
- Architectural placement: positioned logically between client application code and back-end database servers (Resource Managers).
- Thread & Queue Management: Manages client connection pools (client request queues) and worker thread dispatchers.

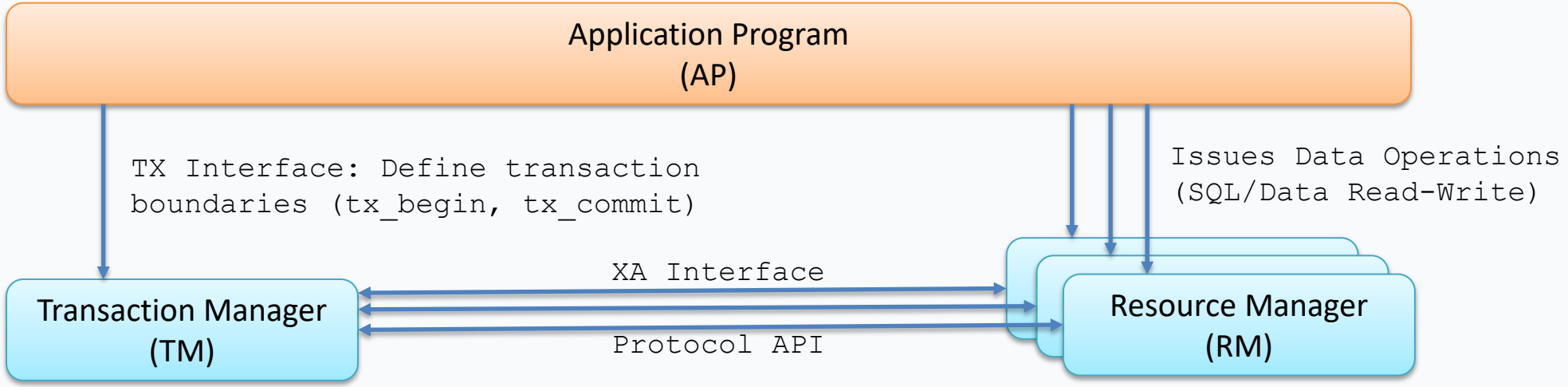
Core Middleware Functions

- **Distributed Commit Coordination**: Enforces 2PC/3PC across multi-vendor databases.
- Standard Protocols: Uses X/Open DTP and XA standards for resource manager interoperability.
- **Fault Recovery**: Intercepts server crashes and automatically drives recovery routines.



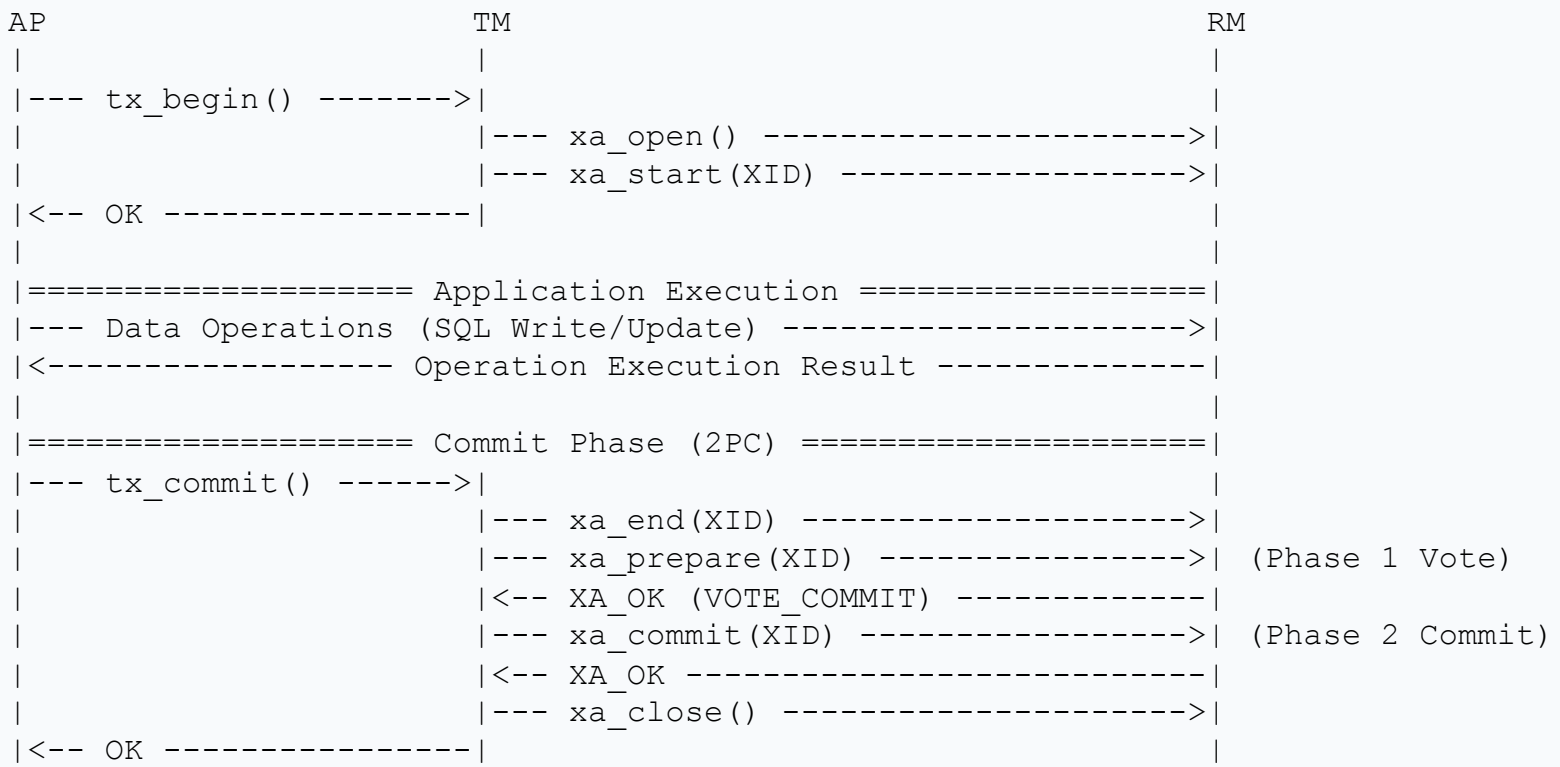
X/Open Distributed Transaction Processing (DTP) model

1. **Application Program (AP)**: Implements business logic and marks transaction scope using standard demarcation calls (`tx_begin()`, `tx_commit()`, `tx_rollback()`).
2. **Transaction Manager (TM)**: Assigns global transaction identifiers (XIDs), tracks transaction branches across distributed storage nodes, coordinates distributed commit protocols, and supervises system recovery.
3. **Resource Manager (RM)**: Manages persistent storage (e.g., relational database systems, transactional message queues) and exposes native transactional access interfaces for local lock execution, logging, and state recovery.



X/Open Distributed Transaction Processing (DTP) model

During a distributed transaction commit, the TM and RM interact using the C-API XA Interface standard:



Distributed Concurrency Control: Strict 2PL

Isolation & Lock Types

- **Concurrency Control**: Prevents read-write and write-write conflicts between concurrent transactions.
- **Shared** Locks (S): Required for reading data; multiple transactions can hold shared locks concurrently.
- **Exclusive** Locks (X): Required for writing data; grants exclusive access to a single transaction.

Strict Two-Phase Locking (Strict 2PL)

- **Growing Phase**: Transaction acquires needed locks as data items are accessed (shared locks for reads and exclusive locks for writes); releases none.
- **Shrinking Phase**: ALL locks are held continuously until the transaction fully commits or aborts.
    - **Cascading Abort Prevention**: Holding locks until commit prevents other transactions from reading uncommitted data.

➔ **Distributed Deadlock Hazard**: Mutual wait dependencies across independent lock tables can create global deadlocks.

Timestamp Ordering & Optimistic Concurrency Control

Timestamp Ordering (TO)

- Transaction Timestamps: Each transaction T_i receives a unique logical timestamp $ts(T_i)$.
- Data Item Timestamps: Items record maximum read timestamp $R(x)$ and write timestamp $W(x)$.
- Ordering Rule: Operations out of timestamp order are rejected, aborting and restarting T_i.
- Pessimistic Overhead: Requires blocking or frequent timestamp checks on every access.

Optimistic Concurrency Control (OCC)

- Conflict Assumption: Assumes data access conflicts are rare in read-heavy workloads.
- **Phase 1 (Read)**: Executes freely in a private workspace without acquiring locks.
- **Phase 2 (Validation)**: Verifies no conflicting writes occurred during execution.
- **Phase 3 (Write/Abort)**: Commits workspace writes to database if valid; otherwise aborts.

Content

- Module 1: Distributed Transactions & Concurrency Control (Slides 01–08)
- Module 2: Fault Tolerance Concepts & Failure Models (Slides 09–16)
- Module 3: Reliable Communication & Distributed Commit Protocols (Slides 17–26)
- Module 4: System Recovery Mechanisms & Logging Strategies (Slides 27–34)
- Module 5: Real-World Systems, Assessment & Practical Lab (Slides 35–40)

Dependability Traits: Availability, Reliability, Safety, Maintainability

Availability vs. Reliability

- Availability: Probability the system is operational at any given instant:

$$A(t) = \frac{MTTF}{MTTF + MTTR}$$

- Reliability: Probability the system operates continuously without failure over time interval Δt.
- Distinction: A system crashing for 1ms every hour has 99.999% availability but low reliability.

Safety & Maintainability

- Safety: Measure of how gracefully a system fails without causing catastrophic real-world damage.
- Maintainability: How easily and quickly a failed system can be repaired (measured by $MTTR$).
- $MTTF$: Mean Time To Failure (uptime duration).
- $MTTR$: Mean Time To Repair (downtime duration).

The Causal Chain: Fault → Error → Failure

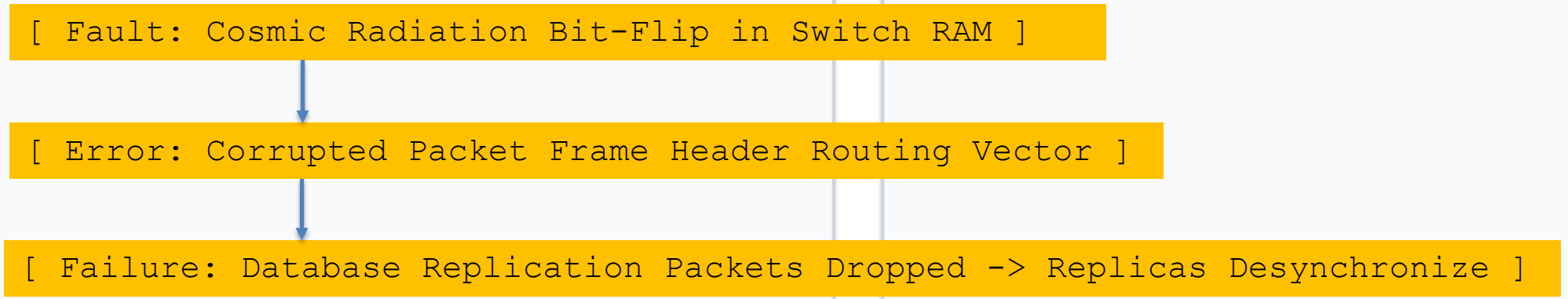
Causal Chain Definitions

- **Fault**: The underlying cause of an invalid state (e.g., bug in code, bad RAM cell, cut fiber cable).
- **Error**: An invalid internal system state resulting from a fault (e.g., corrupted memory bit).
- **Failure**: External behavior deviating from system specifications (e.g., server crash or wrong output).

Fault Classifications

- *Transient Faults*: Occurs once and vanishes (e.g., cosmic ray bit flip).
- *Intermittent Faults*: Recurs sporadically and unpredictably (e.g., loose wiring connection).
- *Permanent Faults*: Persists until the physical component is replaced or repaired.

Cascade Example:



Failure Models Taxonomy in Distributed Systems

Failure Model	System Behavior	Severity Level
Fail-Stop/ Crash-Stop	Server halts permanently; state is lost, but no incorrect operations occur.	Lowest
Crash-Recovery	Server halts, reboots, and recovers using stable write-ahead logs.	Low
Omission Failures	Server fails to respond to incoming requests or drop outgoing messages.	Medium
Timing Failures	Server response is correct in content, but arrives outside time bounds.	High
Arbitrary Failures (Byzantine)	Server outputs malicious, corrupted, or contradictory data to peers.	Highest (Critical)

Detailed Analysis: Omission & Byzantine Failures

Omission Failure Modes

- Receive Omission: A server fails to receive messages sent over the network (e.g., buffer overflow).
- Send Omission: A server fails to transmit messages queued for outgoing network interfaces.
- Channel Omission: Network routers or switches drop packets in transit without application notice.
- Recovery: Non-Byzantine systems assume fail-stop or crash-recovery semantics.

Byzantine Failure Modes

Named after the Byzantine Generals Problem (Lamport, Shostak, Pease, 1982).

- Arbitrary Behavior: Faulty or compromised processes actively output contradictory data.
- Collusion: Faulty nodes can send conflicting state updates to different peers (e.g., Commit to A, Abort to B).
- Cryptographic Defense: Requires digital signatures, message authentication, and strict $(3k + 1)$ quorums.

Failure Masking Strategies & Triple Modular Redundancy (TMR)

Redundancy Paradigms

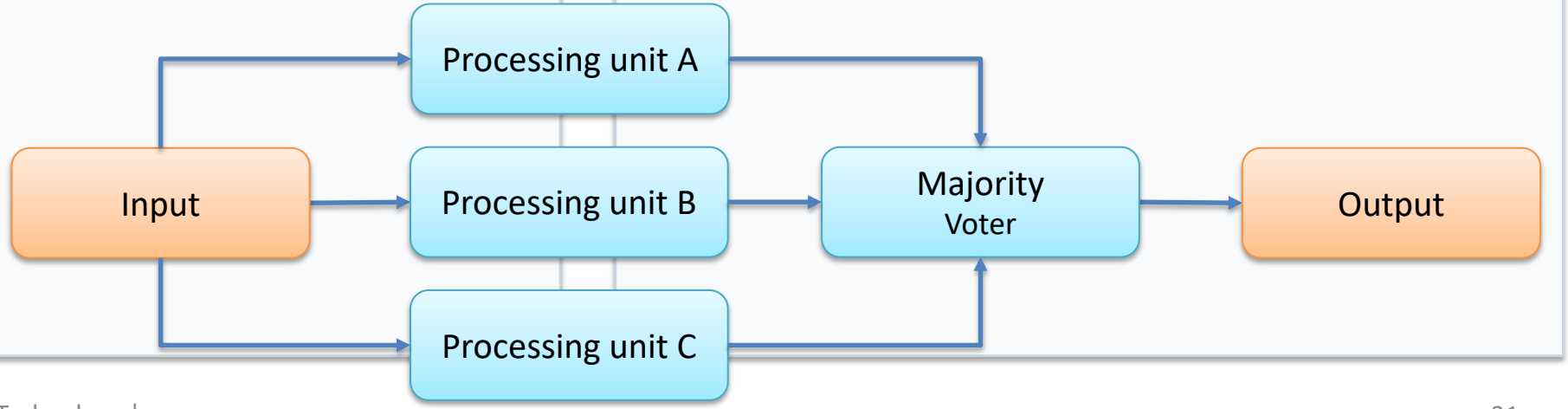
- **Information Redundancy**: Adding extra check bits or ECC code to detect/correct bit corruption.

  (Error-correcting codes/Hamming codes)

- **Temporal Redundancy**: Retrying failed operations or retransmitting dropped packets (e.g., TCP ACK timeouts).
- **Physical Redundancy**: Adding extra hardware units (CPUs, memory, duplicate server nodes).

Triple Modular Redundancy (TMR)

- Architecture: Input signal is split across 3 **identical** processing units (A, B, C).
- **Majority Voter**: Outputs the majority result from the 3 processing units.
- *Fault Masking*: If Unit C fails and outputs 0 while A and B output 1, the voter outputs 1, masking C's failure.



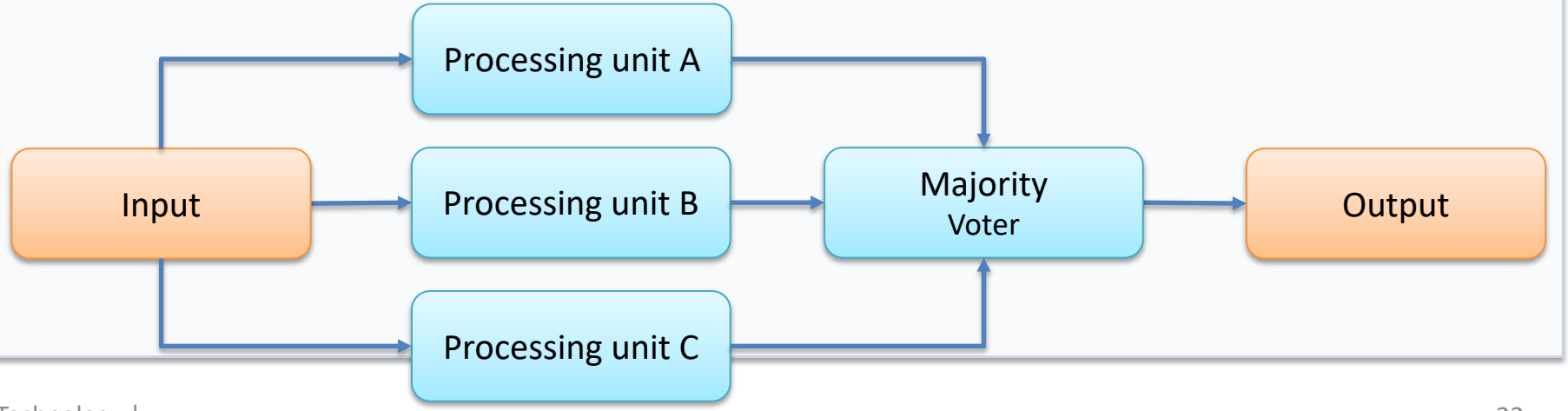
Failure Masking Strategies & Triple Modular Redundancy (TMR)

- **Masking capacity**: TMR masks any single arbitrary failure or complete node crash (1-fault tolerant).
- **Mathematical Reliability Derivation**: Given independent component reliability p (probability of correct operation), the TMR system reliability R_{TMR} is calculated by summing the probabilities of having 3 or 2 operational components:

$$R_{\text{TMR}} = \binom{3}{3} p^3 + \binom{3}{2} p^2(1 - p) = p^3 + 3p^2(1-p) = 3p^2 - 2p^3$$

TMR enhances overall system reliability beyond a single component if $p > 0.5$.

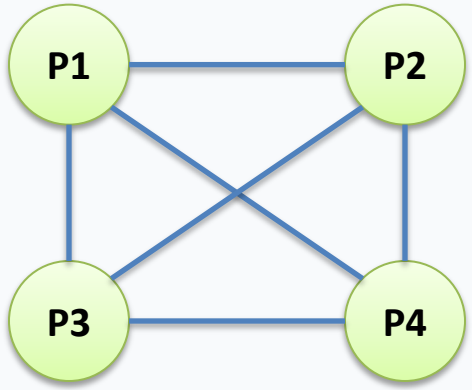
If $p < 0.5$, the voter is more likely to select corrupted outputs.



Process Groups and Replication Organizations

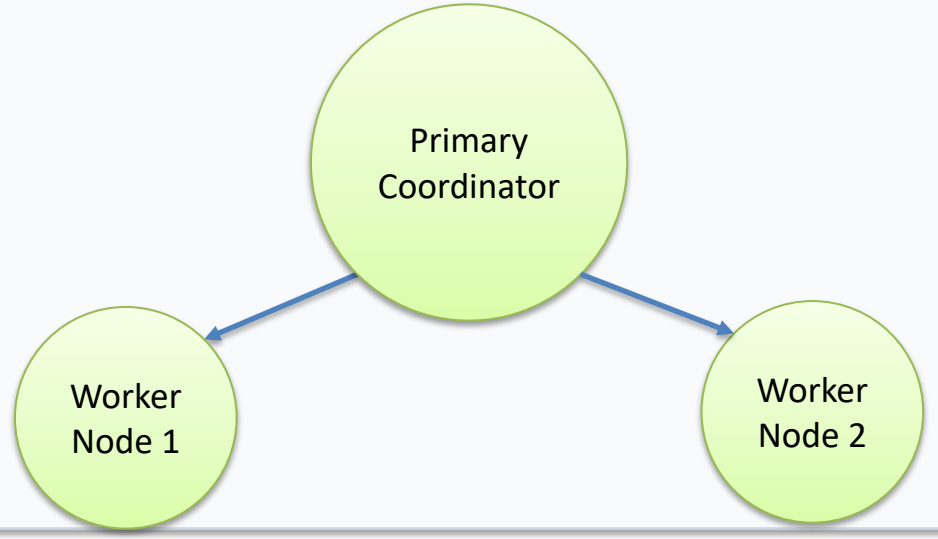
Flat Process Groups

- Peer Architecture: All processes are equal; decisions are reached via peer majority voting.
- Advantage: Highly resilient; no Single Point of Failure (SPOF).
- Disadvantage: High communication complexity ($O(N^2)$ message traffic).



Hierarchical Process Groups

- Master-Worker Architecture: A single Primary Coordinator routes requests to worker replicas.
- Advantage: Fast response time; low message overhead ($O(N)$ traffic).
- Disadvantage: Primary failure stalls the group until a new coordinator is elected.



Process Resilience & k-Fault Tolerance Bounds

Crash-Stop Failure Bound $(k + 1)$

- k-Fault Tolerant Definition: A system that continues operating correctly even if up to k components fail simultaneously.
- Crash-Stop Requirement: To tolerate k crash-stop failures, a system requires at least $k + 1$ process replicas.
- Reasoning: If k nodes crash, 1 operational node remains to produce the correct result.

Byzantine Majority Bound $(2k + 1)$

- Byzantine Fault Definition: Nodes can output corrupted, delayed, or malicious state updates.
- Byzantine Majority Requirement: Requires at least $2k + 1$ process replicas for silent majority voting.
- Reasoning: In a voting pool of $2k + 1$ nodes, non-faulty nodes outvote k faulty nodes.

The Byzantine Generals Limit ($3k + 1$ Requirement)

Byzantine Agreement Bound $(3k + 1)$

- **Contradictory Message Limit**: If faulty nodes output lying or conflicting messages, majority voting fails unless $N \geq 3k + 1$.
- **Minimum Cluster Size**: To tolerate $k = 1$ Byzantine node, a system must have at least $N = 4$ nodes.

Intuition Proof ($N = 3, k = 1$ Impossibility)

- Scenario: 1 Commander, 2 Lieutenants. One node is a traitor.
- Ambiguity: If Commander orders 'Attack' and Lieutenant 1 claims Commander ordered 'Retreat', Lieutenant 2 cannot distinguish who lied.
- Conclusion: Consensus cannot be reached deterministically with 3 nodes if 1 is Byzantine.

Content

- Module 1: Distributed Transactions & Concurrency Control (Slides 01–08)
- Module 2: Fault Tolerance Concepts & Failure Models (Slides 09–16)
- Module 3: Reliable Communication & Distributed Commit Protocols (Slides 17–26)
- Module 4: System Recovery Mechanisms & Logging Strategies (Slides 27–34)
- Module 5: Real-World Systems, Assessment & Practical Lab (Slides 35–40)

Consensus & Agreement in Distributed Systems

The Consensus Problem

- Goal: Getting N independent processes to agree on a single data value or decision over an unreliable network.
- Requirements: Agreement (all non-faulty nodes decide same value), Validity (decided value was proposed), Termination (all nodes reach decision).

FLP Impossibility Theorem (1985)

- Fischer, Lynch, Paterson Result: In an asynchronous network, **NO deterministic consensus protocol** can guarantee safety and liveness if even 1 process can crash.
- Workarounds: Use partial synchrony (timeouts), or randomized/quorum protocols (Paxos, Raft).

Reliable RPC Communication & Orphan Elimination

RPC Failure Modes & Orphan Definition

- RPC Failure Modes: Client lost request, Server crash during processing, Server response lost, Client crash.
- Orphan Process Hazard: A server thread continuing heavy computation for a client that has already crashed or timed out.
- Resource Drain: Orphans consume CPU/RAM and hold active database locks indefinitely.

4 Orphan Elimination Strategies

- **Extermination**: Client logs RPC calls; upon reboot, explicitly kills remote orphan stubs.
- **Reincarnation**: Client broadcasts epoch numbers on reboot; servers kill stubs from older epochs.
- **Gentle Reincarnation**: Servers kill orphans when epoch changes and no active owner responds.
- **Expiration**: Remote stubs receive a time lease; if lease expires without renewal, orphan terminates.

- *Critical Clock Dependency* (Tanenbaum): To prevent a server from prematurely terminating an active lease that a client assumes is valid, the maximum physical clock drift rate ρ across nodes must be tightly bounded:

$$\Delta t(1+\rho) < LeaseDuration$$

Atomic Commit Protocols: Two-Phase Commit (2PC)

2PC Objective & Architecture

- Objective: Ensures all database participants in a distributed transaction either COMMIT or ABORT atomically.
- Architecture: A single Coordinator node coordinates N Participant nodes.

Phase 1 & Phase 2 Mechanics

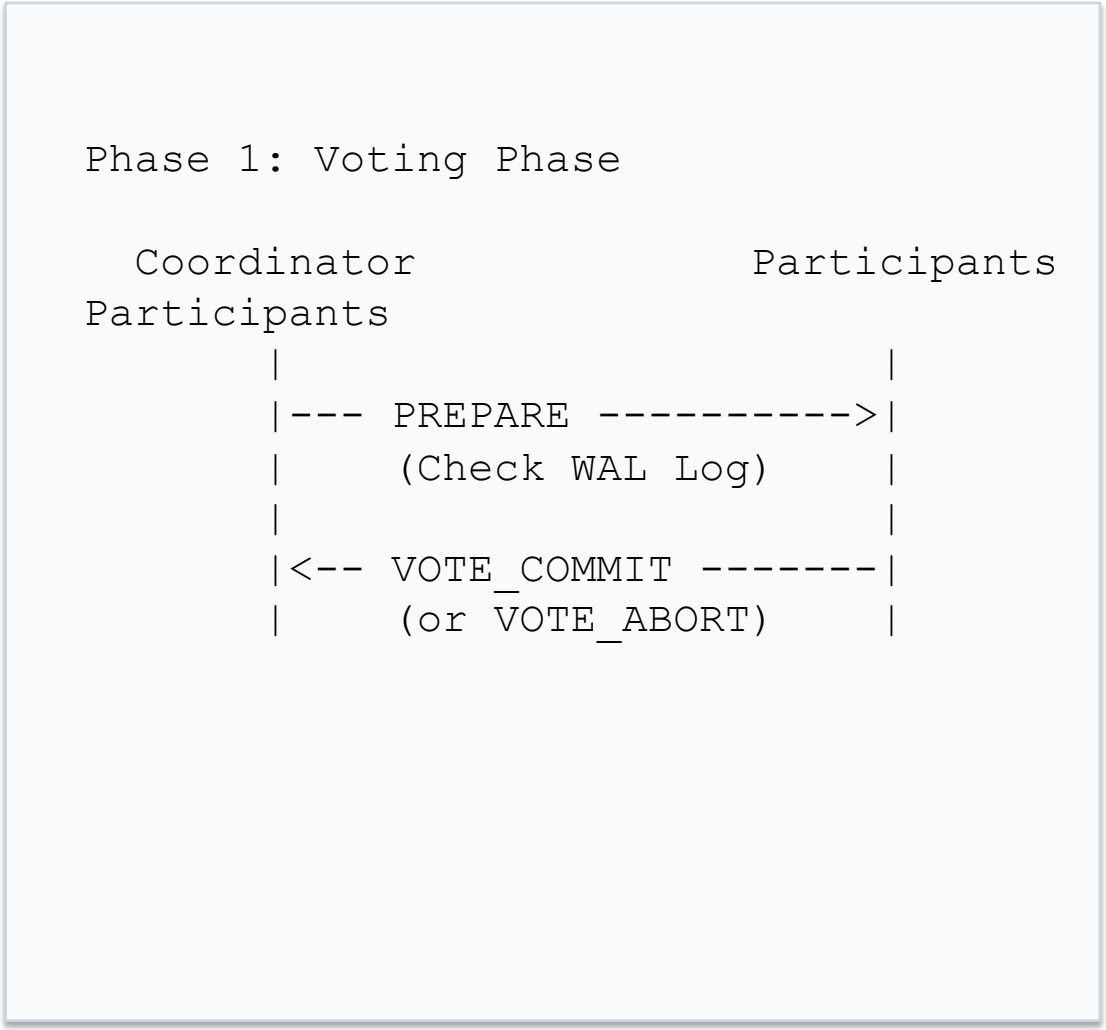
- **Phase 1 (Voting)**:

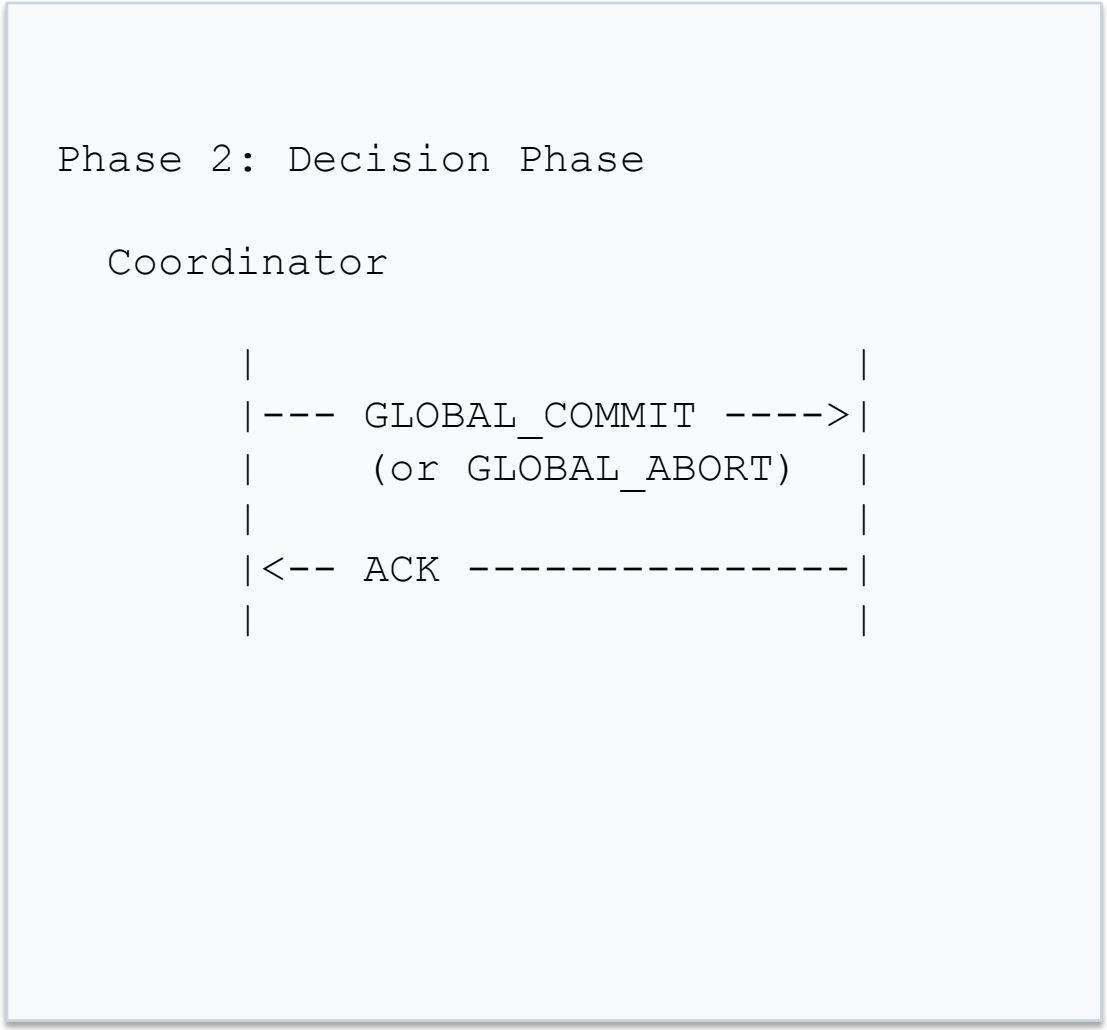
1. Coordinator logs `START_2PC` and multicasts `VOTE_REQUEST` to all participants.
2. Participants check local logs/locks.
   If ready, logs `VOTE_COMMIT` and replies `VOTE_COMMIT`.
   Otherwise, write `VOTE_ABORT` and reply `VOTE_ABORT`.

- **Phase 2 (Decision)**:

1. If ALL vote `COMMIT`, Coordinator logs and multicasts `GLOBAL_COMMIT`.
2. If ANY vote `ABORT` (or timed out), Coordinator logs and multicasts `GLOBAL_ABORT`.
3. Participants write final decision to log and return an `ACK`.

Atomic Commit Protocols: Two-Phase Commit (2PC)

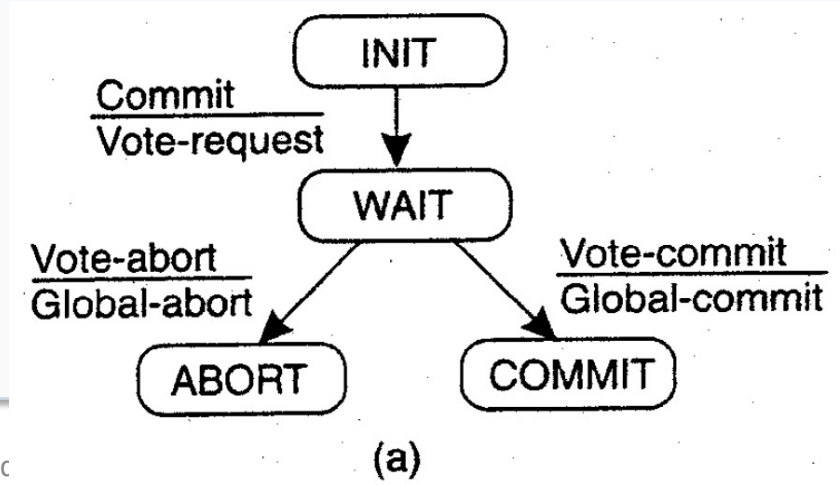




2PC Finite State Machines: Coordinator & Participant

Coordinator State Machine

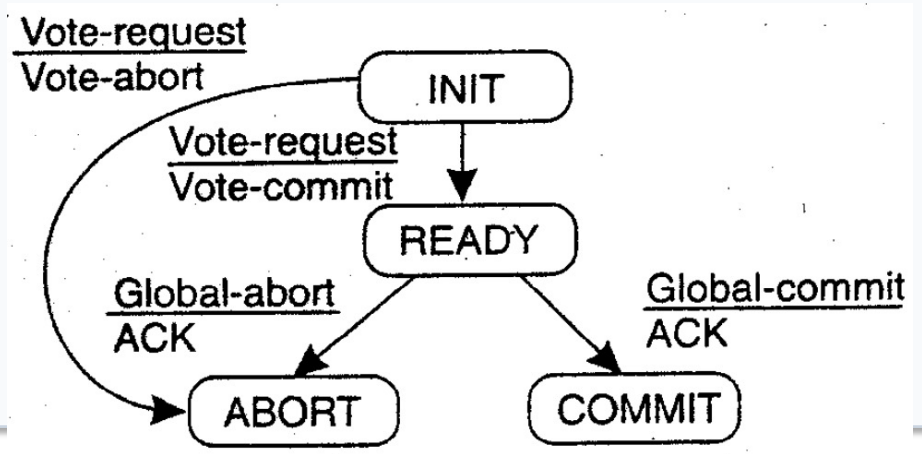
- INIT: Multicast VOTE_REQUEST ⟶ Transition to WAIT state ⟶ Enter WAIT state.
- In WAIT: Receive all VOTE_COMMIT ⟶ Transition to COMMIT ⟶ Log & Multicast GLOBAL_COMMIT.
- In WAIT: Receive any VOTE_ABORT / Timeout ⟶Transition to ABORT ⟶ Log & Multicast GLOBAL_ABORT.



(a)

Participant State Machine

- INIT: Receive VOTE_REQUEST:
  - Vote COMMIT⟶ Log VOTE_COMMIT ⟶ Transition to READY state (Blocked!).
  - Vote ABORT ⟶ Log VOTE_ABORT ⟶ Transition to ABORT state.
- In READY: Wait for GLOBAL_COMMIT or GLOBAL_ABORT from coordinator.



(b)

2PC Crash Recovery Protocols & Write-Ahead Logging

Write-Ahead Logging Rule

- Mandatory: Every state transition must be written to stable storage BEFORE sending network messages.

Crash Recovery Decision Tree

- *Participant crashes in* INIT: Reboots, finds no log entry ⟹ Sends local ABORT.
- *Participant crashes in* READY: Reboots, finds VOTE_COMMIT ⟹ Must query Coordinator or peers to learn decision.
- *Coordinator crashes in* WAIT: Reboots, finds START_2PC but no decision ⟹ Multicasts GLOBAL_ABORT.
- *Coordinator crashes in* COMMIT: Reboots, finds GLOBAL_COMMIT ⟹ Re-transmits GLOBAL_COMMIT to participants.

The Fundamental Flaw of 2PC: Blocking Architecture

The 2PC Blocking Scenario

- Coordinator sends VOTE_REQUEST. All participants vote COMMIT and enter READY state.
- Coordinator crashes BEFORE writing or transmitting the global decision.
- Simultaneous Failure: A participant node also crashes.

Why Participants Block Indefinitely

- Remaining participants in READY state cannot communicate among themselves to resolve the state.
- Cannot Abort: Crashed participant and coordinator might have committed.
- Cannot Commit: Crashed participant might have voted ABORT.

Consequence: Participants remain blocked indefinitely, holding active database locks!

Non-Blocking Commit: Three-Phase Commit (3PC)

3PC Objective & Innovation

- Objective: Eliminate the blocking vulnerability of 2PC under fail-stop assumptions.

- Innovation: Introduces an intermediate PREPREPARE state between READY and COMMIT.

3PC Protocol Phases

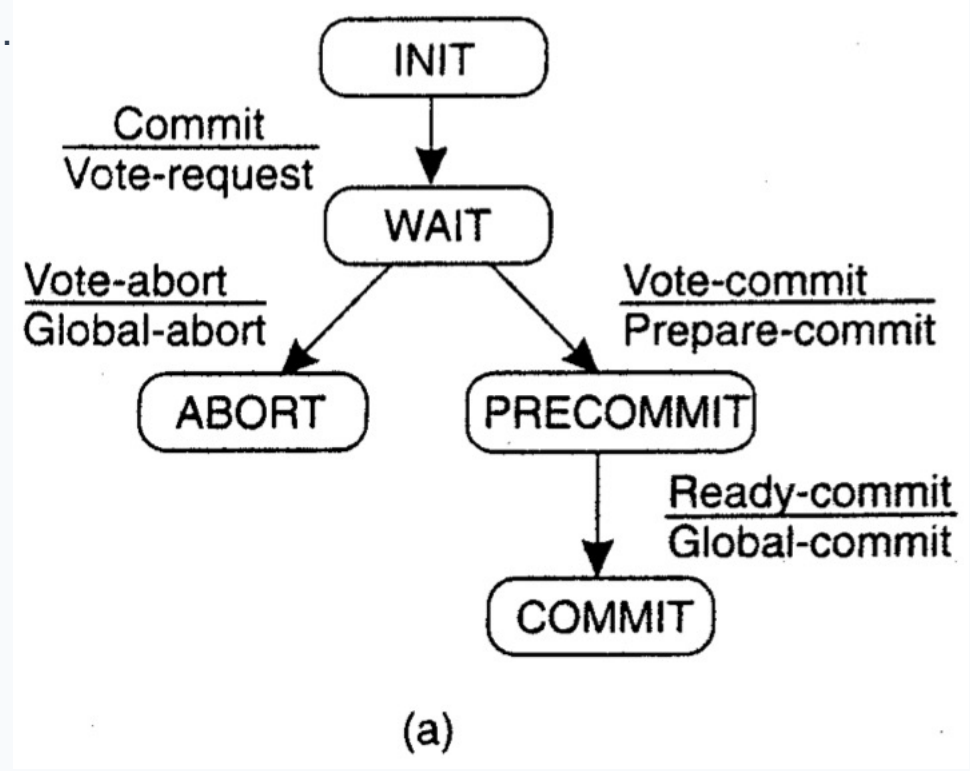
- ***Phase 1 (Voting)***: Coordinator sends VOTE_REQUEST; participants reply VOTE_COMMIT or VOTE_ABORT.

- ***Phase 2 (Pre-Commit)***: If all vote COMMIT, Coordinator multicasts PREPREPARE. Participants transition to PREPREPARE and reply ACK.

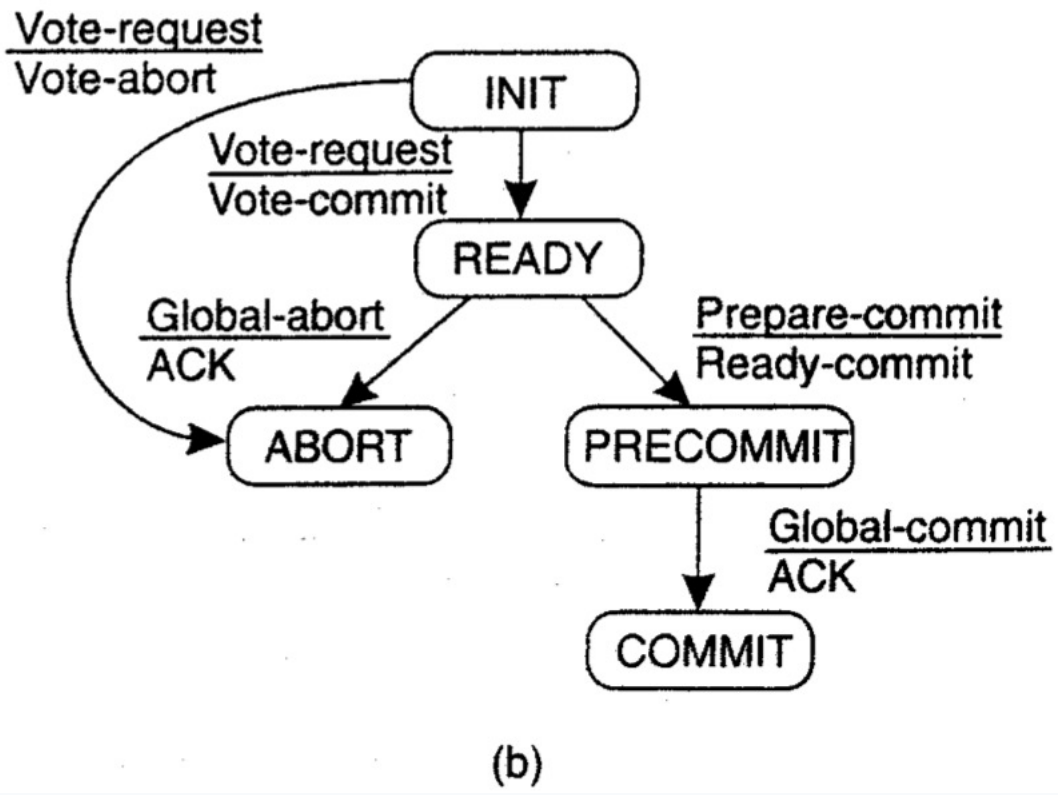
- ***Phase 3 (Final Commit)***: Upon receiving all ACKs, Coordinator multicasts GLOBAL_COMMIT. Participants commit.

Non-Blocking Commit: Three-Phase Commit (3PC)

Finite state machine for the coordinator



Finite state machine for a participant



3PC Recovery Rules & Real-World Limitations

3PC Non-Blocking Recovery Rules

- If coordinator crashes, remaining participants elect a new coordinator.
- Rule A: If at least one participant has reached PREPREPARE, it is safe to proceed to COMMIT.
- Rule B: If no participant has passed READY state, the new coordinator safely defaults to GLOBAL_ABORT.

Real-World Limitation

- Fail-Stop Assumption: 3PC is non-blocking ONLY under strict fail-stop bounds.
- Network Partition Hazard: If a network split occurs, 3PC can produce split-brain conflicting commits!
- Result: Modern cloud stores replace 3PC with consensus commit logs (Paxos/Raft).

2PC vs. 3PC Invariant & State Comparison

Feature / Dimension	Two-Phase Commit (2PC)	Three-Phase Commit (3PC)
Phases	2 (Voting, Decision)	3 (Voting, Pre-Commit, Decision)
Participant States	INIT, READY, COMMIT, ABORT	INIT, READY, PREPREPARE, COMMIT, ABORT
Blocking Property	Blocking (Coordinator failure blocks READY nodes)	Non-blocking under Fail-Stop model
Message Rounds	2 rounds (4 message steps)	3 rounds (6 message steps)
Network Partition	Safe (Blocks, preserves consistency)	Unsafe (Can produce inconsistent commits)

Consensus Commit Protocols (Paxos & Raft)

Limitations of Traditional Commit Protocols

- 2PC blocks if coordinator crashes; 3PC fails under network partitions.
- Need: A protocol that handles both coordinator crashes AND asynchronous network partitions safely.

Paxos and Raft Consensus Engine Replacements

- Replicated State Machines: Replaces single coordinators with a majority consensus group.
- Quorum Commit: Writes succeed as long as a majority (floor(N/2) + 1) of replicas log the proposal.
- Automatic Failover: Raft/Paxos automatically elects a new leader without stalling active transactions.

Content

- Module 1: Distributed Transactions & Concurrency Control (Slides 01–08)
- Module 2: Fault Tolerance Concepts & Failure Models (Slides 09–16)
- Module 3: Reliable Communication & Distributed Commit Protocols (Slides 17–26)
- Module 4: System Recovery Mechanisms & Logging Strategies (Slides 27–34)
- Module 5: Real-World Systems, Assessment & Practical Lab (Slides 35–40)

System Recovery Paradigms: Backward vs. Forward Recovery

Backward Recovery

- Mechanism: Restores the system from a corrupted state back to a previously saved, valid historical state (**checkpoint**).
- *Advantage*: Generic and application-independent.
- *Disadvantage*: High performance overhead; discards all computation performed since the checkpoint.

Forward Recovery

- Mechanism: Moves the system forward from an erroneous state into a new valid operational state.
- *Advantage*: Discards zero valid computation.
- *Disadvantage*: Requires domain-specific knowledge to correct errors on the fly (e.g., self-correcting codes).

Distributed Checkpointing: Independent vs. Coordinated

Independent Checkpointing

- Mechanism: Each process saves its local state to stable storage whenever convenient.

- *Risk*: Cascading rollbacks during recovery (**The Domino Effect**).

Coordinated Checkpointing

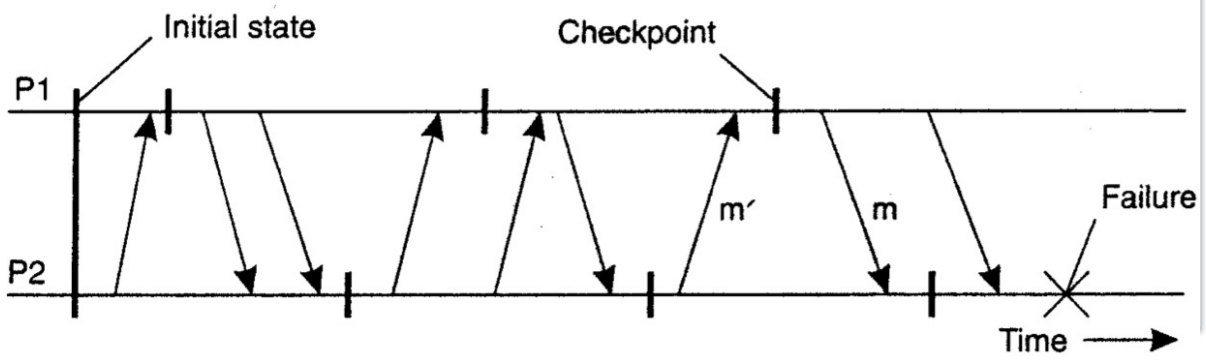
- Mechanism: Processes synchronize using network messages to record a globally consistent system state (**Recovery Line**).

- *Chandy-Lamport Algorithm*: Uses marker messages to capture process states and in-flight channel messages without halting execution.

The Domino Effect & Distributed Recovery Lines

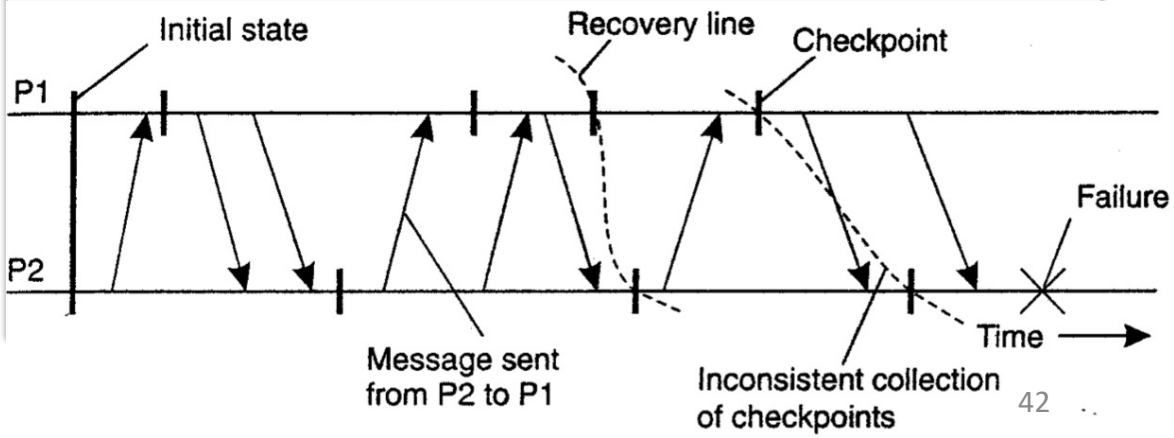
The Domino Effect Mechanism

- Trigger: Occurs during backward recovery with uncoordinated independent checkpoints.
- Cascading Rollback: If a message dependency exists between P_1 and P_2, rolling back P_1 forces P_2 to roll back prior to sending the message.
- Catastrophic Outcome: Cascading dependency rollbacks push the system all the way back to initial boot state!



Recovery Line Definition

- Consistent Global State: The most recent set of local process checkpoints that forms a valid system state.
- No Orphan Messages: Contains no messages recorded as received that were not recorded as sent.



The Chandy-Lamport Distributed Snapshot Algorithm

Algorithm Objective

- Computes a consistent global state (Recovery Line) without halting process execution.

Marker Protocol Rules

1. *Marker Transmission*: Process P_i saves its local state and sends a MARKER message along all outgoing channels.

2. *Marker Reception (First Time)*: When P_j receives MARKER from channel C: saves local state, records C as empty, and forwards MARKER along outgoing channels.

3. *Marker Reception (Subsequent)*: Records all incoming messages on channel C as channel state until MARKER arrives on C.

Message Logging & Deterministic Replay

Piecewise Deterministic (PWD) Assumption

- System Execution Model: System execution consists of deterministic state transitions separated by non-deterministic events (e.g., message arrivals).

- Deterministic Replay: Replaying the same sequence of incoming messages to a checkpointed state reproduces the exact same final state.

Message Logging Concept

- Combines periodic checkpointing with logging incoming messages to avoid rolling back to old checkpoints.

Pessimistic Message Logging Mechanics

Pessimistic Logging Execution

- Synchronous Logging: Synchronously writes every incoming message to stable storage BEFORE delivering it to the application layer.

Trade-Off Analysis

- *Benefit*: Recovery is simple and localized; orphan processes are completely impossible.
- *Drawback*: High I/O performance penalty on every single incoming message.
- **Use Case**: Mission-critical safety systems where zero recovery complexity is required.

Optimistic Message Logging Mechanics & Orphan Elimination

Optimistic Logging Execution

- Asynchronous Logging: Buffers incoming messages in memory and asynchronously flushes log batches to disk in the background.

- *Benefit*: High throughput; zero network message delay penalty.

Orphan Process Handling

- Orphan Risk: A server crash before memory buffer flushes creates orphan processes on surviving peers.

- Orphan Elimination: On recovery, the system inspects dependency vectors and rolls back orphan processes to align state.

Logging Strategy Comparison: Pessimistic vs. Optimistic

Dimension / Feature	Pessimistic Message Logging	Optimistic Message Logging
Disk I/O Logging	Synchronous (Before message delivery)	Asynchronous (Background memory buffer batching)
Runtime Latency	High message latency overhead	Low latency / High throughput
Orphan Processes	Completely impossible	Possible upon server crashes
Recovery Complexity	Low (Replay logged messages from last checkpoint)	High (Requires dependency vector tracking & orphan rollbacks)
Primary Use Case	Safety-critical & financial transaction processing	High-performance distributed computing

Content

- Module 1: Distributed Transactions & Concurrency Control (Slides 01–08)
- Module 2: Fault Tolerance Concepts & Failure Models (Slides 09–16)
- Module 3: Reliable Communication & Distributed Commit Protocols (Slides 17–26)
- Module 4: System Recovery Mechanisms & Logging Strategies (Slides 27–34)
- Module 5: Real-World Systems, Assessment & Practical Lab (Slides 35–40)

Real-World Case Study: X/Open DTP & XA Standard Interface

Enterprise Role

- Cross-database distributed transactions in enterprise software (Oracle, IBM DB2, PostgreSQL).

The XA Interface Standard

- Standardizes the interface between Transaction Manager (TP Monitor) and Resource Managers (Databases).

- API Function Calls: xa_open(), xa_start(), xa_prepare() [2PC Phase 1], xa_commit() [2PC Phase 2], xa_rollback().

- Ubiquity: Java JTA/JDB and C++ enterprise frameworks use XA 2PC under the hood.

Real-World Case Study: Consensus Commit Logs in Raft & Paxos

Cloud Infrastructure Role

- Modern cloud-native key-value stores (etcd, HashiCorp Consul, CockroachDB).

Replacing 2PC/3PC with Replicated State Machines

- Replaces single transaction coordinators with a majority consensus group.
- Writes succeed as long as a majority quorum (floor(N/2) + 1) of replicas log the proposal.
- Automatic leader reelection eliminates manual crash recovery stalls in Kubernetes (etcd).

Numerical Review Problems & Mathematical Calculations

Problem 1: k-Fault Tolerance Replication Bounds

- Scenario: A cluster must survive up to k = 2 simultaneous failures.
- Crash-Stop Bound: N = k + 1 = 2 + 1 = 3 process replicas.
- Byzantine Bound: N = 3k + 1 = 3(2) + 1 = 7 process replicas.

Problem 2: System Availability Calculation

- Given: MTTF = 990 hours, MTTR = 10 hours.
- Formula: Availability A = MTTF / (MTTF + MTTR)
- Calculation: A = 990 / (990 + 10) = 990 / 1000 = 99.0% Availability.

Conceptual & Analytical Review Questions

Self-Assessment Question 1

- Explain the precise combination of node failures that causes Two-Phase Commit (2PC) to block indefinitely.

- Model Answer: Coordinator crashes in WAIT state after participants vote COMMIT, and a participant also crashes.

Self-Assessment Question 2

- Why does independent checkpointing risk causing the Domino Effect during backward recovery?

- Model Answer: Message dependencies across unaligned checkpoints cause cascading rollbacks all the way to boot.

Practical Programming Lab: 2PC Simulator

```
// 2PC Coordinator Pseudocode

def execute_2pc_transaction(tx_id, participants):

    log.append(f'START_2PC {tx_id}')

    votes = multicast('VOTE_REQUEST', participants)

    if all(v == 'VOTE_COMMIT' for v in votes.values()):

        log.append(f'GLOBAL_COMMIT {tx_id}')

        multicast('GLOBAL_COMMIT', participants)

    else:

        log.append(f'GLOBAL_ABORT {tx_id}')

        multicast('GLOBAL_ABORT', participants)
```

Key Implementation Details

- Language: Python, Java, or Go.
- Architecture: 1 Coordinator socket server + 3 Participant socket servers.
- Stable Storage: Synchronously append state changes to local WAL log files.
- Failure Injection: Simulate a Participant voting ABORT and reboot log recovery.

Lab Expected Execution Trace & Evaluation Rubric

```
// Sample Terminal Trace Output

[COORD] Tx 101 Started -> Log START_2PC

[COORD] Multicast VOTE_REQUEST

[P1] Log VOTE_COMMIT -> Reply VOTE_COMMIT

[P2] Log VOTE_COMMIT -> Reply VOTE_COMMIT

[P3] Lock Conflict! -> Log VOTE_ABORT -> Reply VOTE_ABORT

[COORD] Recv VOTE_ABORT -> Log GLOBAL_ABORT

[COORD] Multicast GLOBAL_ABORT

[P1] Recv GLOBAL_ABORT -> Rollback -> Reply ACK

[SYSTEM] Tx 101 Aborted Cleanly.
```

Key Implementation Details

- 2PC Protocol & State Machines: 40%
- Stable Storage Write-Ahead Logging: 30%
- Failure Injection & Log Recovery Test: 20%
- Design Report & Documentation: 10%